计算机设计与实践

流水线CPU及SoC设计

任务A：理想流水线设计

2025 · 夏

HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- 加深对流水线CPU**结构**和**工作原理**的理解
- 熟练掌握数字电路的仿真调试方法

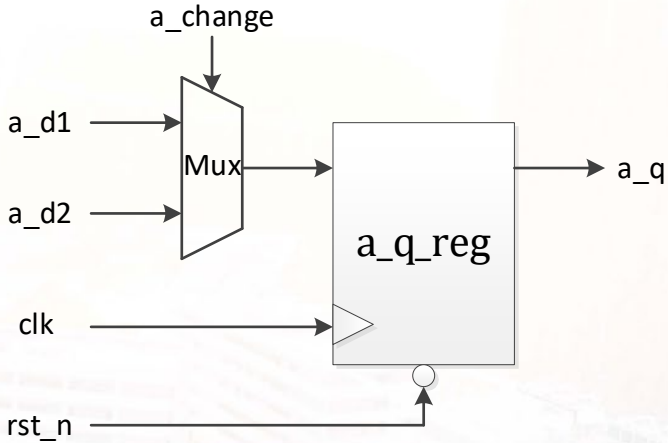
实验内容

- 根据CPU的功能，划分流水线
- 根据流水线的划分添加流水寄存器，实现理想流水线
- 运行功能仿真，验证理想流水线CPU

回顾

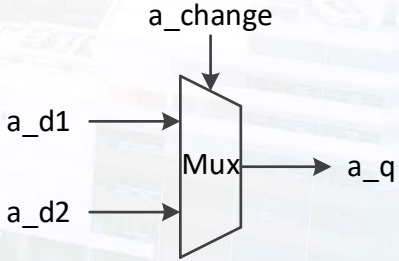
- 时序逻辑

```
reg [3:0] a_q;

always @ (posedge clk or negedge rst_n) begin
    if (~rst_n)         a_q <= 4'h0;
    else if (a_change)  a_q <= a_d1;
    else                a_q <= a_d2;
end
```



- 组合逻辑

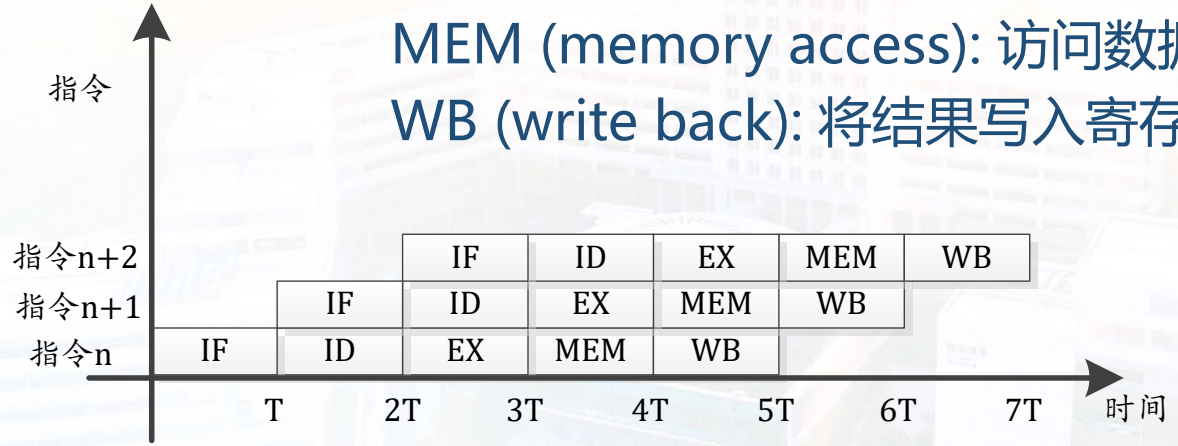
```
reg [3:0] a_q;

always @ (*) begin
    if (a_change)  a_q = a_d1;
    else           a_q = a_d2;
end
```



流水线划分

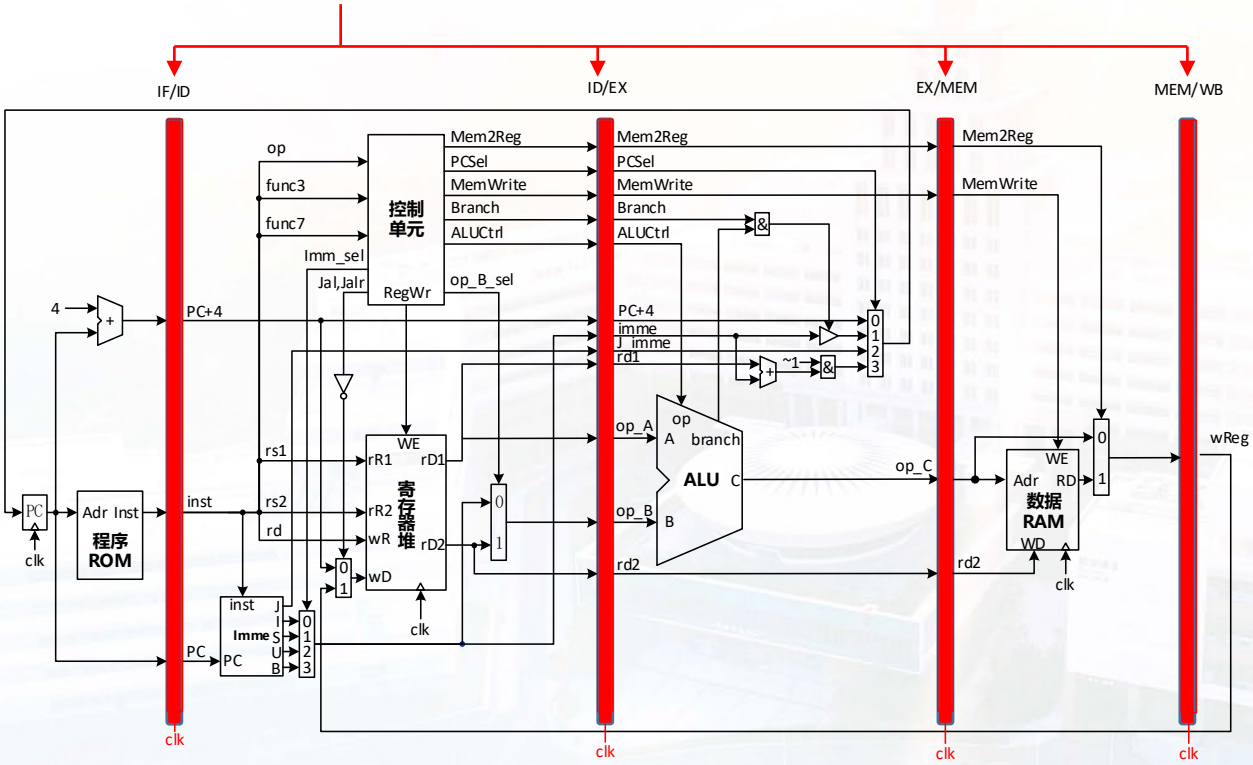
- 五级流水线

IF (instruction fetch): 从存储器中取出指令
ID (instruction decode): 读寄存器并译码指令
EX (execute): 执行操作或计算地址
MEM (memory access): 访问数据存储器中的操作数
WB (write back): 将结果写入寄存器



流水寄存器

- **添加流水寄存器**



1）保存该流水级输出数据信息，交给下一级处理，并共享给其他指令

2）切割组合逻辑，提升系统频率

流水寄存器

- **确认流水寄存器内容（IF/ID）**

PC：J型指令、B型指令

PC+4： J型指令

inst：所有指令

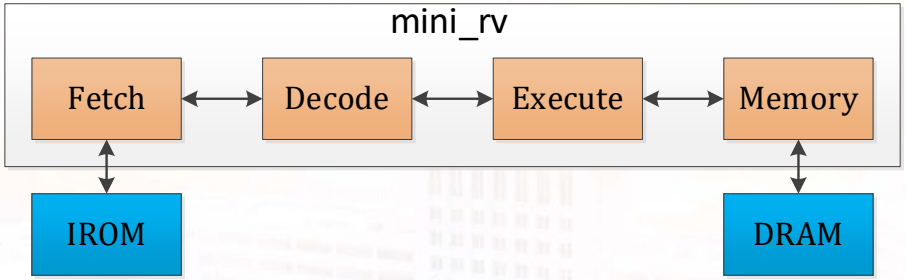
RTL实现（示例）

```
always @ (posedge clk or posedge rst) begin
    if (rst) id_pc <= 32'h0;
    else     id_pc <= if_pc;
end

always @ (posedge clk or posedge rst) begin
    if (rst) id_inst <= 32'h0;
    else     id_inst <= if_inst;
end
```

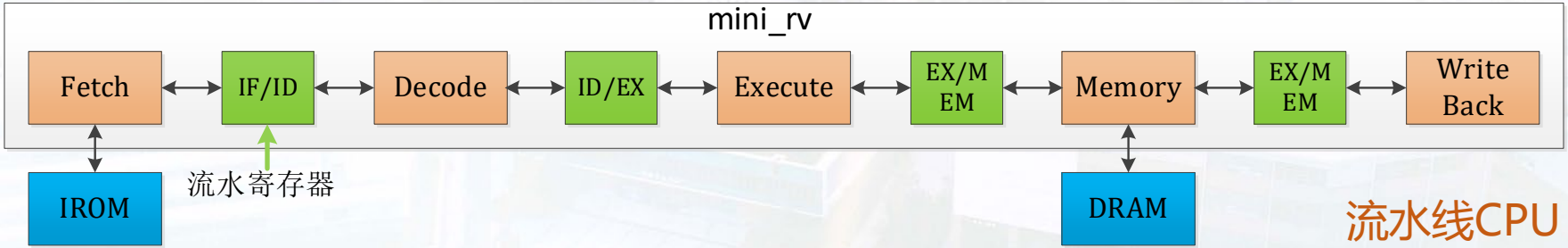
流水寄存器

- **添加流水寄存器的框图**

示例



单周期CPU



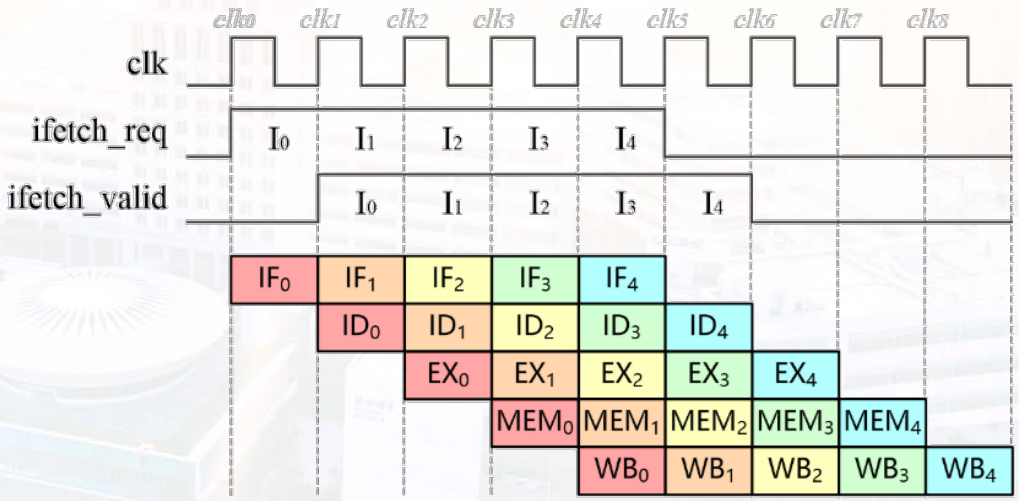
流水线CPU

取指逻辑

- 修改取指逻辑：从复位撤销开始，每个时钟周期不间断连续取指

```
1 // 复位信号发生边沿变化时首次取指；上一条已取回，同时立即取下一条
2 assign ifetch_req  = first_req | ifetch_valid;
3 assign ifetch_addr = pc;
```

- 指令何时处于IF阶段?
  - 指令发出取指请求时，指令就处于IF阶段
- 指令何时处于ID阶段?
  - 指令机器码返回时，指令可以进行译码了，指令就处于ID阶段



理想流水线仿真验证

- 使用指导书提供的指令序列，在RARS汇编

```
xori    t0, zero, 1
ori     t1, zero, 2
addi    t2, zero, 3
lui     t4, 0x4
auipc   t5, 0x0
slti    t6, zero, 2
add     s0, t0, t1
```

- 导出机器码，生成.coe文件并导入至IROM
- 运行功能仿真，观察指令能否重叠执行
- **注意代码规范性，可以避开很多问题！**

开始实验